

Reverse Engineering GitHub Copilot in VS Code

What We Learned



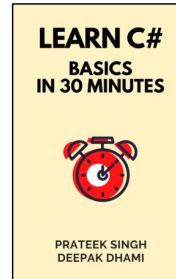
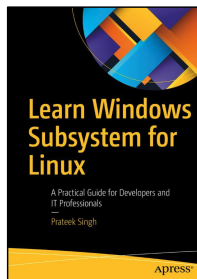


Prateek Singh

Founder @ ganak.ai
Ex-Staff SWE LinkedIn



Books



Articles



Course



INTERACTIVE COURSE

Learning Python through PowerShell Concepts

Beginner 75 Lessons 4h 40min Certificate of Completion

Linkedin Newsletter

NEWSLETTER

Beyond The Code with Prateek

Your weekly edge in Software, AI, Cloud and key learnings that keep you ahead. Subscribe now!

By **Prateek Singh**
Founder @ ganak.ai / IIT Roorkee / ex-LinkedIn / 5x ...

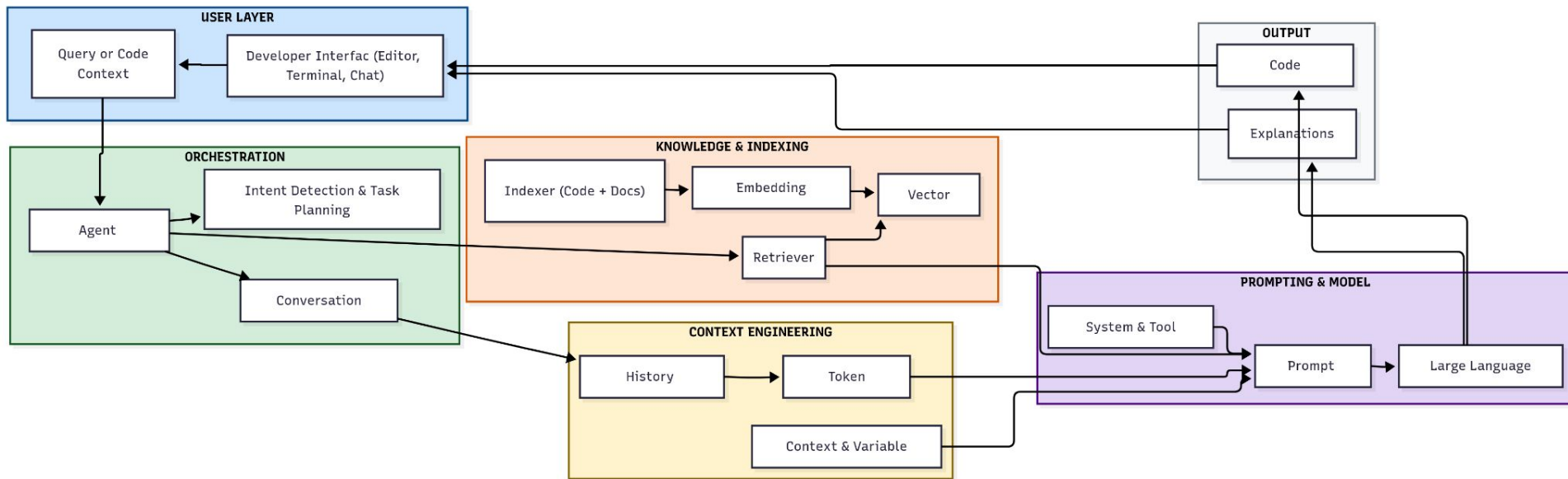
Published weekly
1,409 subscribers

Agenda



1. **Anatomy of an AI Coding Assistant – A Primer**
2. **How Github Copilot in VS Code works?**
A 10,000 foot overview!
3. Let's dig in! – Proof is in the code
4. Key Learnings
5. References

Anatomy of an AI Coding Assistant



How Github Copilot works?

GitHub Copilot's architecture rests on three main components:

1. Copilot Extension (on VS Code Local)

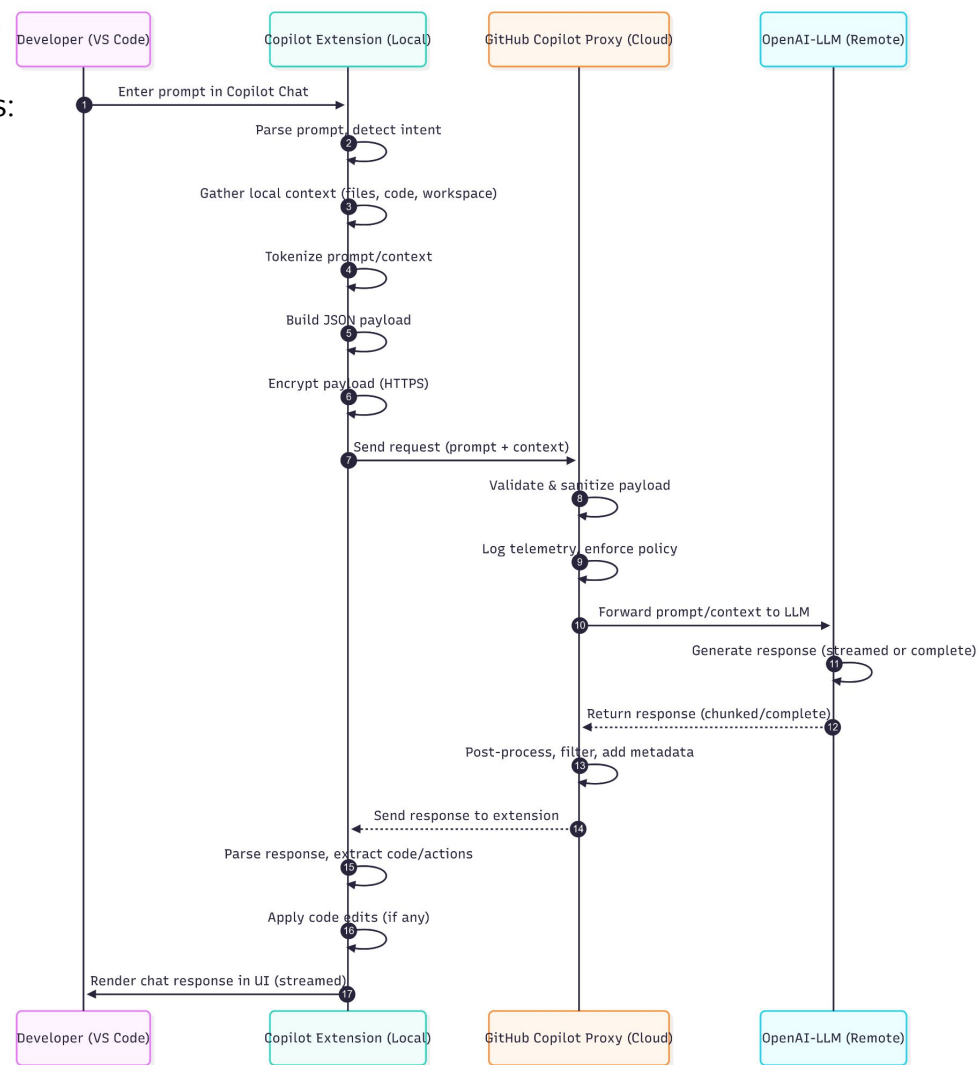
- The **local component** that **captures your prompt**, **identifies relevant code** from your workspace
- **Formats the data** before sending it out.

2. Github Copilot Proxy (Cloud)

- Sits **between your local extension and the OpenAI backend**.
- Handles **rate-limiting, authentication, security** checks, and forwarding requests to/from the model. These are often **referred to as filters**.

3. Backend LLM (Remote)

- The **large language model (LLM)** that **processes your prompt (plus relevant context)**
- Returns AI-generated suggestions or explanations.



DEMO - How Github Copilot works?



1. Gathering Context

- a. Intent – Agent, Inline, Terminal
- b. Finding Allowed Tools
- c. Tokenization & Budgets

2. Composing Prompts – [vscode-prompt-tsx](#)

3. Calling Backend Proxy

- a. Safety & Governance
- b. Observable traces: chat transcript, edits, session logs.

4. LLM Generates Response

5. Streaming Response Chunks to VS Code

Key Learnings & Takeaways



1. **Context is a superpower**
 - a. Local indexing, Minimal snippets, Summarize/trim
2. **Prompts are software**
 - a. Declarative, Composable (TSX/JSX), System/Instr/User split , Diffable
3. **Layered instruction strategy**
 - a. Safety → Identity → Format → Task , Predictable outputs
4. **Multi-agent orchestration**
 - a. Specialized participants , Intent routing , Fewer retries
5. **Tool-augmented loops**
 - a. Propose → Execute tools → Diff/patch , Human review
6. **Networking & UX**
 - a. One round-trip/turn , Streaming chunks , Token hygiene/early stop
7. **Security & compliance**
 - a. TLS/HSTS , Content exclusion , Least-privilege tools , Review gates

References

- **What are participants?**
 - [Craft language model prompts | Visual Studio Code Extension API](#)
- **Prompt-TSX**
 - [Craft language model prompts | Visual Studio Code Extension API](#)
- [How github copilot works](#) **[Short video]**
- [GitHub Copilot Chat Explained: The Life of a Prompt](#)
- [Pseudo XML prompt recommendation](#) **[Anthropic]**
- [The technology behind GitHub's new code search](#)
- **How AI Code Editor works**
 - [The anatomy of an AI coding assistant | Sourcegraph Blog](#)
 - [Anatomy of an Open Source AI Coding Agent - Joel's Blog](#)

Thank you & Let's keep the conversation going!

SUBSCRIBE TO MY NEWSLETTER

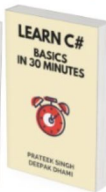
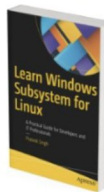


NEWSLETTER

Beyond The Code with Prateek

Your weekly edge in Software, AI, Cloud
and key learnings that keep you ahead.
Subscribe now!

READ MY BOOKS



Prateek Singh ✓ 🔊

Founder @ ganak.ai / IIT Roorkee / ex-LinkedIn / 5x Author / Speaker on
AI & Cloud

India · [Contact info](#)

7,968 followers · 500+ connections



Ganak AI Labs



Indian Institute of
Technology, Roorkee



Connect with me on LinkedIn

I share insights on AI, Cloud, and Software Engineering.